CORE103 Computational Thinking
PW4 - Iteration



Department of Computer Science

Dhanani School of Science and Engineering

Habib University

Fall 2026

Copyright @ 2026 Habib University

Contents

1 Introduction **2**
1.1 Instructions 2
1.1.1 Lab Objectives 2

2 Prerequisites **3**
2.1 Assert Statements 3
2.2 Pytest 3

3 Exercises **5**
3.1 Odd Number Sum 5
3.2 Prime Number Check 6
3.3 Triangular Number Sequence 7
3.4 Coding Challenge Score 9
3.5 Lucky Raffle Ticket 10
3.6 War in the Milky Way 11
3.7 Fruits Landing on a House 13

Introduction

1.1 Instructions

- This PW is **UNGRADED**. However, it is highly encouraged that you attempt and submit it to get feedback from your section tutor.
- This PW must be submitted online via CANVAS. You are required to submit a zip file that contains all the `.py` files.
- The zip file should be named as `PW4_aa123456.zip` where `aa123456` will be replaced with your student id.

1.1.1 Lab Objectives

The following are the lab objectives of this lab:

- Iteration in Python

Prerequisites

2.1 Assert Statements

Assert statements are crucial tools in programming used to verify assumptions and test conditions within code. For instance, in Python, an assert statement is typically as follows:

```
assert condition, message
```

For example, in a function calculating the square root, in order to ensure that the input value is non-negative, one might add the following assert statement:

```
assert x >= 0, 'x should be a non-negative number'
```

If the condition evaluates to False, an AssertionError is raised, highlighting the issue and its corresponding message.

2.2 Pytest

Pytest is a popular testing framework in Python known for its simplicity and power in facilitating efficient and scalable testing for applications and software. In order to use Pytest, one first needs to install it and create test functions prefixed with *test_* in a Python file. To install Pytest, you can write the following command on the terminal:

```
pip install pytest
```

After installing Pytest, one can execute all the tests by typing the following command on the terminal:

```
pytest
```

If you want to test any particular file, then you will have to specify the filename as follows:

```
pytest <filename>.py
```

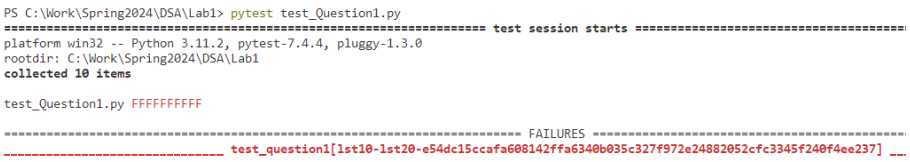
If all the tests pass successfully, then you should receive a similar output.

```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
============================= test session starts ========
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py ..........

============================== 10 passed in 0.06s =========
```

However, if one or more tests fail, you should receive a similar output.



Exercises

3.1 Odd Number Sum

This question should be implemented in the file ***q1.py****.*

A student society assigns consecutive numbered tokens during an event. For one activity, only odd-numbered tokens contribute points equal to their token number. Given two token numbers, first_number and second_number, determine the total points contributed by all odd-numbered tokens between them, including both endpoints.

The values may be provided in either order. For example, if first_number is greater than second_number, the function must still consider all numbers between them.

Function Specification

Write an **iterative** function named **sum_odd** that takes two parameters, **first_number** and **second_number**, and returns the sum of all odd integers between them, inclusive.

If there are no odd integers between the two values, the function must return `0`.

Input Validation

There are no numerical constraints on first_number and second_number. However, both arguments must be integers.

If either argument is not an integer, the function must return:

```
'Error: bad argument. sum_odd is defined for integers only.'
```

Type Checking

You may use Python's built-in `type()` function to check whether an argument is an integer.

```
>>> type(3) == int
True

>>> type(3.14) == int
False
```

Sample Interaction

```
>>> sum_odd(1, 1)
1

>>> sum_odd(2, 2)
0

>>> sum_odd(1, 2)
1

>>> sum_odd(2, 3)
3

>>> sum_odd(13, 2)
48

>>> sum_odd(6.7, 10)
'Error: bad argument. sum_odd is defined for integers only.'
```

For `sum_odd(1, 1)`, the only number in the range is 1, which is odd, so the function returns 1.

For `sum_odd(2, 2)`, there are no odd numbers in the range, so the function returns 0.

For `sum_odd(13, 2)`, the odd numbers between 2 and 13 are $3, 5, 7, 9, 11$, and 13. Their sum is 48, so the function returns 48.

Testing

To test your function, type the following command in the terminal:

```
pytest tests/test_q1.py
```

3.2 Prime Number Check

This question should be implemented in the file ***q2.py****.*

During a campus coding activity, students are given positive integers and must determine whether each number is a **prime number**. A prime number is a positive integer greater than 1 that is divisible only by 1 and itself.

Function Specification

Write an **iterative** function named **is_prime** that takes one integer parameter, number. The function must return `True` if number is divisible only by 1 and itself; otherwise, it must return `False`.

Constraints

- `2 <= number <= 200`

Sample Interaction

```
>>> is_prime(3)
True

>>> is_prime(8)
False
```

For `number = 3`, the only positive divisors are 1 and 3, so the function returns `True`.

For `number = 8`, the number is also divisible by 2 and 4, so it is not prime and the function returns `False`.

Testing

To test your function, type the following command in the terminal:

```
pytest tests/test_q2.py
```

3.3 Triangular Number Sequence

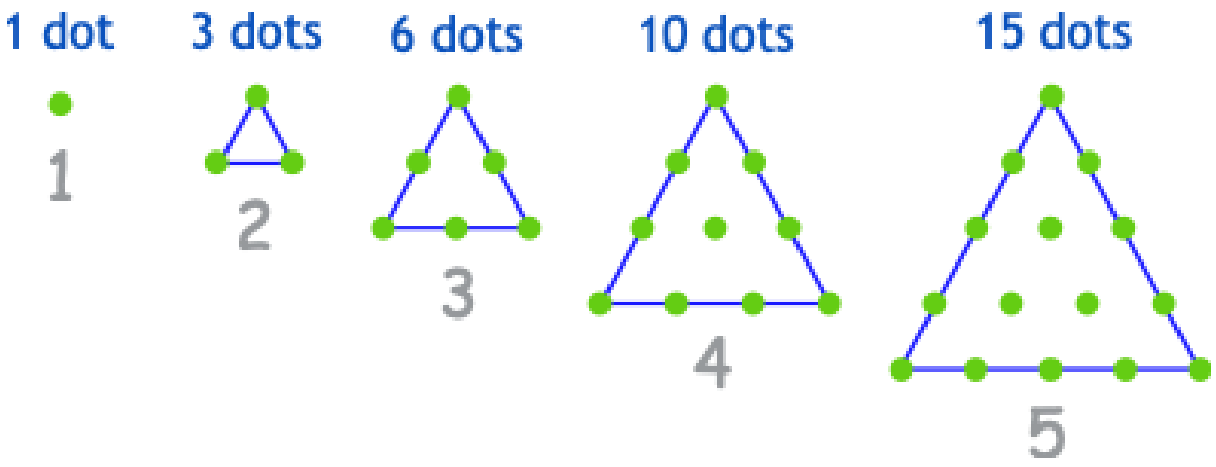
This question should be implemented in the file ***`q3.py`***.

The **triangular number sequence** begins as follows:

```
1, 3, 6, 10, 15, 21, 28, 36, 45, ...
```

Each term represents the number of dots that can be arranged to form a triangle. The first term contains 1 dot, the second term contains 3 dots, the third term contains 6 dots, and so on.

Each new term can be obtained by adding the next positive integer to the previous term.



Function Specification

Write an **iterative** function named **triangle_number_sequence** that takes one integer parameter, **term_count**. The function must print the first term_count terms of the triangular number sequence.

The entire sequence must be printed on **one line**, with consecutive terms separated by a comma followed by one space. Do not print a trailing comma after the final term.

Constraints

- term_count is an integer.
- `1 <= term_count <= 20`

Sample Interaction

```
>>> triangle_number_sequence(1)
1

>>> triangle_number_sequence(4)
1, 3, 6, 10
```

For `term_count = 1`, only the first term of the sequence is printed.

For `term_count = 4`, the first four triangular numbers are $1, 3, 6$, and 10, so the function prints:

```
1, 3, 6, 10
```

Output Formatting

Python's `print()` function normally moves to a new line after printing. You may use the `end` parameter to control what is printed after a value.

```
print(1, end=', ')
print(2, end=', ')
print(3)
```

This produces:

```
1, 2, 3
```

Testing

To test your function, type the following command in the terminal:

```
pytest tests/test_q3.py
```

3.4 Coding Challenge Score

This question should be implemented in the file ***q4.py****.*

During a campus coding challenge, each submission is tested against total_tests test cases.

The result of each test case is recorded in a string test_results, where `1` means the test case passed and `0` means it failed.

The score for the submission is determined using the following rules:

- If all total_tests test cases are passed, the submission receives `100` points.
- Otherwise, if the first required_tests test cases are all passed, the submission receives partial_score points.
- Otherwise, the submission receives `0` points.

Function Specification

Write an **iterative** function named **submission_score** that takes three integer parameters: **total_tests**, **required_tests** and **partial_score**, and one string parameter, **test_results**.

The parameter test_results is a string containing exactly total_tests characters. Each character is either `'0'` or `'1'`. The function must return the score earned according to the rules given above.

Constraints

- `1 <= required_tests < total_tests <= 10`
- `1 <= partial_score < 100`
- test_results contains exactly total_tests characters.
- Every character in test_results is either `'0'` or `'1'`.

Sample Interaction

```
>>> submission_score(4, 2, 50, '1011')
0

>>> submission_score(3, 2, 66, '110')
66

>>> submission_score(4, 2, 50, '1111')
100
```

For `test_results = '1011'`, not all 4 test cases are passed, and the second of the first 2 test cases failed. Therefore, the function returns `0`.

For `test_results = '110'`, not all 3 test cases are passed, but the first 2 test cases are passed. Therefore, the function returns `66`.

For `test_results = '1111'`, all 4 test cases are passed, so the function returns `100`.

Testing

To test your function, type the following command in the terminal:

```
pytest tests/test_q4.py
```

3.5 Lucky Raffle Ticket

This question should be implemented in the file ***`q5.py`***.

During a university carnival, the raffle organizers use a special rule to decide whether a ticket is lucky. A ticket is considered `Lucky` if its digits can be rearranged to form an integer divisible by 5. Otherwise, the ticket is considered `Not Lucky`.

Function Specification

Write an **iterative** function named **lucky_ticket** that takes one positive integer parameter, **ticket_number**. The function must determine whether the digits of ticket_number can be rearranged to form an integer divisible by 5.

The function must:

- return `"Yes"` if such a rearrangement is possible;
- return `"No"` otherwise.

Constraints

- `1 <= ticket_number <= 100000`
- **Solve this question arithmetically using iteration.**
- **You are not allowed to convert ticket_number to a string.**

Sample Interaction

```
>>> lucky_ticket(101)
'Yes'

>>> lucky_ticket(119)
'No'
```

For `ticket_number = 101`, its digits can be rearranged to form `110`, which is divisible by 5. Therefore, the function returns `"Yes"`.

For `ticket_number = 119`, its digits cannot be rearranged to form an integer divisible by 5, so the function returns `"No"`.

Testing

To test your function, type the following command in the terminal:

```
pytest tests/test_q5.py
```

3.6 War in the Milky Way

This question should be implemented in the file ***q6.py****.*

In the game *War in the Milky Way*, the galaxy contains planet_count planets numbered consecutively from `0` to `planet_count - 1`. Each planet is 1 km away from the next planet. The galaxy is not circular, so planet `0` is not connected to planet `planet_count - 1`.

Two possible power source locations, power_source_1 and power_source_2, are given. For every planet, determine its distance to the nearest valid power source. The goal is to find the **maximum** of these nearest distances.

Function Specification

Write an **iterative** function named **max_power_distance** that takes three integer parameters, **planet_count**, **power_source_1**, and **power_source_2**, and returns the maximum distance from any planet to its nearest valid power source.

A power source is valid only if its planet number is between `0` and `planet_count - 1`, inclusive.

- If both power sources are valid, use the nearer of the two for each planet.
- If only one power source is valid, use that power source.
- If neither power source is valid, return `0`.

Constraints

- `1 <= planet_count <= 100`
- `0 <= power_source_1 <= 100`
- `0 <= power_source_2 <= 100`

Hint

Python's built-in `abs()`, `min()`, and `max()` functions may be useful.

```
>>> abs(3 - 7)
4

>>> min(4, 9)
4

>>> max(4, 9)
9
```

The `abs()` function can be used to find the distance between two planet numbers.

The `min()` function can help determine the nearer of two power sources, while `max()` can help keep track of the largest distance found.

Sample Interaction

```
>>> max_power_distance(5, 0, 4)
2

>>> max_power_distance(3, 6, 7)
0
```

For `max_power_distance(5, 0, 4)`, planets 0 and 4 contain power sources. Planets 1 and 3 are 1 km away from their nearest power source, while planet 2 is 2 km away from either power source. Therefore, the maximum distance is 2 km, so the function returns `2`.

For `max_power_distance(3, 6, 7)`, the valid planets are numbered `0`, `1`, and `2`. Therefore, neither 6 nor 7 is a valid power source location, so the function returns `0`.

Testing

To test your function, type the following command in the terminal:

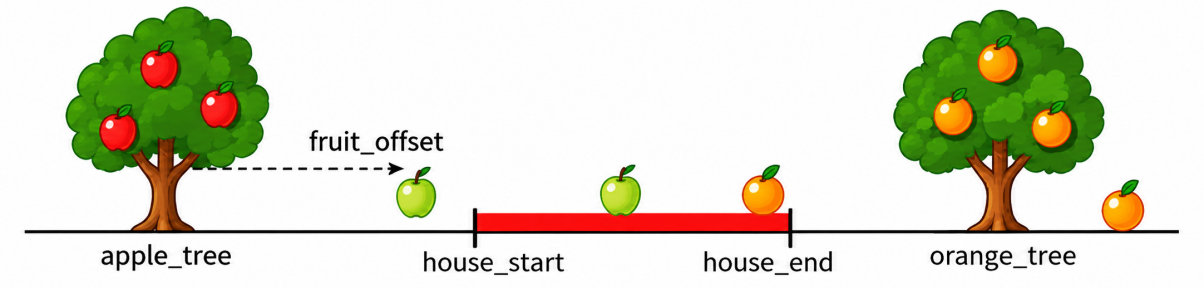
```
pytest tests/test_q6.py
```

3.7 Fruits Landing on a House

This question should be implemented in the file ***`q7.py`***.

Mohammad's house lies on a number line from position house_start to position house_end, inclusive. An apple tree is located at position apple_tree, to the left of the house, and an orange tree is located at position orange_tree, to the right of the house.

When a fruit falls from a tree, its position relative to the tree is represented by an integer fruit_offset. A positive offset means that the fruit lands to the right of the tree, a negative offset means that it lands to the left, and an offset of `0` means that it lands at the tree's position.



For example, if the apple tree is at position `5` and an apple has a fruit_offset of `-2`, then the apple lands at position:

$$5 + (-2) = 3$$

Your task is to determine how many apples and oranges land on Mohammad's house.

Function Specifications

1. Write an **iterative** function named **apples_in_house** that takes four integer parameters: **house_start**, **house_end**, **apple_tree**, and **total_apples**.

   The function must read exactly total_apples integer input values. Each input represents the fruit_offset of one apple from the apple tree at position apple_tree.

   For each apple, calculate its landing position and count the apple if its landing position lies between house_start and house_end, inclusive.

   The function must **return** the number of apples that land on the house.

2. Write an **iterative** function named **oranges_in_house** that takes four integer parameters: **house_start**, **house_end**, **orange_tree**, and **total_oranges**.

   The function must read exactly total_oranges integer input values. Each input represents the fruit_offset of one orange from the orange tree at position orange_tree.

   For each orange, calculate its landing position and count the orange if its landing position lies between house_start and house_end, inclusive.

   The function must **return** the number of oranges that land on the house.

3. Write a main function named **fruits_in_house** that takes six integer parameters: **house_start**, **house_end**, **apple_tree**, **orange_tree**, **total_apples**, and **total_oranges**.

   The function must call **apples_in_house** and **oranges_in_house** to determine the number of apples and oranges that land on the house.

   It must print the two counts on **one line**, separated by one space. Print the apple count first, followed by the orange count.

Constraints

- `1 <= house_start <= 1000`
- `1 <= house_end <= 1000`
- `1 <= apple_tree <= 1000`
- `1 <= orange_tree <= 1000`
- `1 <= total_apples <= 1000`
- `1 <= total_oranges <= 1000`
- fruit_offset is an integer.
- `-1000 <= fruit_offset <= 1000`
- `apple_tree < house_start < house_end < orange_tree`

Interaction

The values house_start, house_end, apple_tree, orange_tree, total_apples, and total_oranges are supplied as arguments when **fruits_in_house** is called.

During the function call:

- **apples_in_house** reads exactly total_apples integer fruit_offset values, one for each apple;
- **oranges_in_house** then reads exactly total_oranges integer fruit_offset values, one for each orange.

Therefore, a total of `total_apples + total_oranges` input values are read.

Sample Interaction

In the sample below, lines written as `[Input -> value]` represent values entered by the user while **fruits_in_house** is running. They are **not** output produced by the function.

```
>>> fruits_in_house(7, 11, 5, 15, 3, 2)
[Input -> -2]
[Input -> 2]
[Input -> 1]
[Input -> 5]
[Input -> -6]
1 1
```

The apple tree is at position `5`. The three apples land at:

$$5 + (-2) = 3, \qquad 5 + 2 = 7, \qquad 5 + 1 = 6$$

Only the apple at position `7` lands on the house.

The orange tree is at position `15`. The two oranges land at:

$$15 + 5 = 20, \qquad 15 + (-6) = 9$$

Only the orange at position `9` lands on the house. Therefore, one apple and one orange land on the house, so the function prints:

```
1 1
```

Testing

To test your function, type the following command in the terminal:

```
pytest tests/test_q7.py
```